

Quantigence: A Multi-Agent Framework for Post-Quantum Security Analysis on Commodity Hardware

Abdulmalik Alquwayfli

af.alquwayfli@gmail.com

Saudi Data & AI Authority, Riyadh, Saudi Arabia

Abstract

The migration to post-quantum cryptography (PQC) forces security teams to synthesize a fast-moving literature spanning lattice theory, implementation security, and shifting NIST policy. We present **Quantigence**, a multi-agent framework that structures this analysis as a supervisor that decomposes a query into sub-tasks and dispatches them to four specialist agents (cryptographic analysis, threat modeling, standards compliance, and risk assessment), each grounded in live external tools: the arXiv and NVD APIs and a local retrieval index over the NIST PQC standards. Agents run serially on a single commodity GPU using a 4-bit-quantized 9B model, keeping the peak memory footprint within an 8 GB budget. The framework computes a Quantum-Adjusted Risk Score (QARS), an automated operationalization of the risk model of Grigaliūnas and Brūzgienė [6], to prioritize assets. On a 40-query benchmark of atomic, machine-checkable questions, tool access is what raises accuracy, from 57% (zero-shot) to 98% (single agent), and a single tool-using agent is, as expected, a strong baseline that multi-agent decomposition does not beat on single-fact lookups. On a second benchmark of ten complex, multi-faceted queries (the setting orchestration is designed for), the supervisor/worker decomposition instead improves rubric coverage from 78% to 89% and judged answer quality. We report both, and further measure robustness to retrieval-corpus poisoning and behavior across model scales. The implementation and benchmark are released as open source.

Code and data: <https://github.com/AbdulmalikDS/quantigence>

Keywords: Multi-Agent Systems · Post-Quantum Cryptography · Agentic AI · Quantum Risk Assessment · Retrieval Poisoning

1 Introduction

The security of modern digital infrastructure rests on the classical hardness of integer factorization (RSA) and the discrete logarithm problem (elliptic-curve cryptography). Shor’s algorithm solves both in polynomial time on a sufficiently large fault-tolerant quantum computer [21], and Grover’s algorithm gives a quadratic speedup against symmetric primitives, effectively halving their security level [7]. The timeline for such a cryptographically-relevant quantum computer (CRQC) is uncertain, but the “harvest-now, decrypt-later” model means data intercepted today is already at risk if its confidentiality must outlast the arrival of a CRQC, the condition formalized by Mosca’s inequality [9]. Migration to post-quantum cryptography (PQC) is therefore a present concern, not a future one.

The migration is complicated by the pace and breadth of the field. NIST finalized its first PQC standards in August 2024 (FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), and FIPS 205 (SLH-DSA) [10–12]) and published a draft transition roadmap, NIST IR 8547, that proposes deprecating 112-bit-security algorithms around 2030 and disallowing quantum-vulnerable algorithms such as

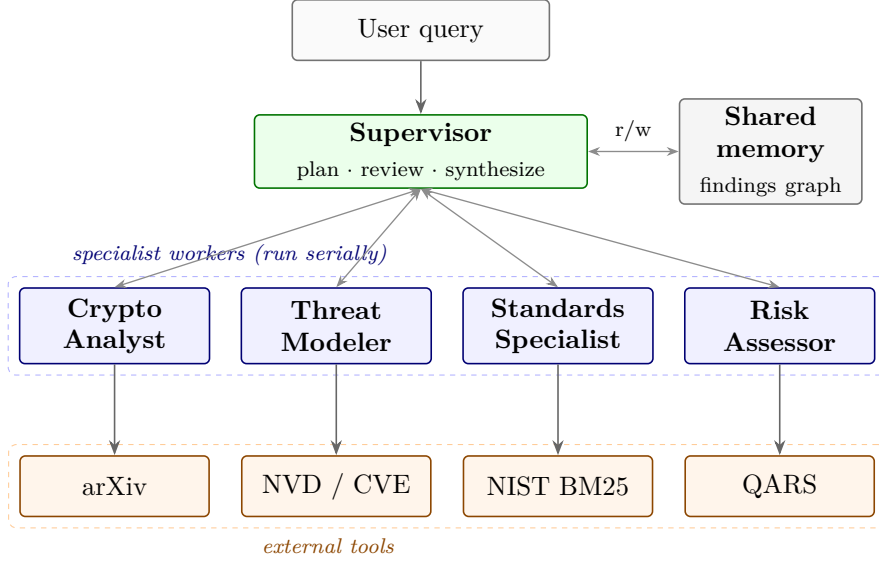


Figure 1: Quantigence architecture. The supervisor plans a query into sub-tasks, dispatches each to a specialist worker (double arrows carry the dispatch and the review), and reads and writes a shared memory of findings when synthesizing. Each worker grounds its reasoning in one external tool: arXiv, the NVD, a local BM25 index over the NIST standards, or the QARS calculator. Workers run serially so a single model instance fits the 8 GB budget (§5).

RSA and ECC around 2035 [13]. At the same time, candidate algorithms have failed late in evaluation, most visibly the isogeny-based scheme SIKE. A sound risk assessment must integrate theoretical cryptanalysis, implementation-level vulnerabilities, standardization policy, and hardware constraints, domains that rarely sit within one analyst’s expertise.

Large language models can help, but a single zero-shot model is a poor fit for this task: it hallucinates citations, loses track of long documents, and cannot easily ground its claims in authoritative sources. Agentic architectures, in which a model plans, calls tools, and reviews its own output, address these weaknesses [20, 23]. Decomposing a research task across specialized agents further improves depth and reduces context interference, as demonstrated by industrial multi-agent research systems [2].

Contributions. This paper makes the following contributions.

1. **A working multi-agent framework for PQC analysis.** We implement a supervisor/worker system with four domain-specialized agents, grounded in live arXiv/NVD APIs and a local NIST retrieval index. The orchestration is a compact, framework-free Python control loop; §2 explains why this matches how deployed security agents are actually built.
2. **An automated Quantum-Adjusted Risk Score.** We operationalize the QARS risk model of Grigaliūnas and Brūzgienė [6] by adding a continuous sigmoid urgency mapping over Mosca’s inequality and wiring parameter estimation into the agents, so the score is computed from retrieved evidence rather than filled in by hand.
3. **An 8 GB-budget execution model.** We show the system runs within an 8 GB VRAM budget on a single GPU using a 4-bit 9B model and serialized agent execution, and we measure how accuracy and memory trade off across the 4B/9B/14B scale.
4. **An honest, reproducible evaluation.** We evaluate with two benchmarks (atomic machine-checkable questions, and complex multi-faceted tasks scored by coverage and a judge), a three-condition ablation on each, a study of robustness to retrieval-corpus poisoning, and full release

of the code and data. We report the negative result (multi-agent does not help atomic lookups) as plainly as the positive one.

2 Related Work

Multi-agent and deep-research systems. Recent “deep research” agents fall into three families: monolithic policies trained end-to-end (OpenAI Deep Research), planner–executor pipelines (Gemini Deep Research; STORM [18]), and orchestrator–worker systems in which a lead agent decomposes a task and dispatches workers with independent context [2]. Quantigence is an instance of the last family, specialized for quantum security and augmented with domain tools and defenses. Agent Laboratory [17] and related systems apply the specialized-agents-per-stage pattern to autonomous science; we adopt the same decomposition but keep a human in the loop for a high-stakes advisory task.

How security agents are built. A survey of contemporary security-focused LLM agents, covering pentesting systems such as PentestGPT [4] and CAI [8] and the agent scaffolds used in benchmarks such as Cybench [24] and CVE-Bench [28], shows that almost none are built on the mainstream agent frameworks; their orchestration layers are hand-written Python (CAI, notably, implements its own lightweight orchestrator). We follow this convention: our control flow is fixed (decompose, dispatch, review, synthesize), so a transparent ~ 300 -line loop is both simpler and more faithful to the paper’s claims than a general-purpose framework would be. We caution that benchmark results for exploitation are distinct from repair: two different benchmarks share the “CVE-Bench” name [22, 28].

Quantum risk scoring. Mosca’s inequality $X + Y > Z$ classifies an asset as at-risk but is binary [9]. Grigaliūnas and Brūzgienė [6] extend it into a multi-factor *Quantum-Adjusted Risk Score* over timeline, sensitivity, and exposure; the concurrent Crypto-Agility Readiness Score (CARS) [16] scores organizational migration readiness on a five-dimension index. Our contribution is not a new score but the automation of QARS: continuous urgency saturation plus agent-driven parameter estimation from retrieved evidence. Other PQC+LLM work targets code migration [14], static auditing [19], and engineering-process visions [26]; to our knowledge Quantigence is the only multi-agent *analysis* assistant for this domain.

Poisoning and prompt injection. Because our agents read untrusted external text, they inherit the attack surface studied by PoisonedRAG [29] (corpus poisoning), AgentDojo and Agent Security Bench [3, 25] (tool-output injection), and defenses such as consistency filtering [27]. We adopt their methodology in §6.4, reporting retrieval and end-to-end attack success separately, and utility under attack.

3 The Quantigence Framework

Quantigence models PQC analysis as a supervised traversal of a shared knowledge store (Figure 1). It has three parts: role specialization, a plan–dispatch–review control loop, and a tool layer that grounds every agent in verifiable data.

3.1 Roles

A *supervisor* plans and integrates; it never performs the detailed analysis itself. Four *worker* personas each carry a distinct system prompt and tool set:

- **Cryptographic Analyst:** lattice, hash-based, and isogeny schemes and their cryptanalysis; primed for rigor.
- **Threat Modeler:** side channels, weak RNGs, and library CVEs; queries the NVD.
- **Standards Specialist:** FIPS 203/204/205 and the IR 8547 draft; retrieves from the local NIST index before answering.
- **Risk Assessor:** estimates Mosca parameters and computes QARS.

3.2 Control Loop

The supervisor decomposes the query into a small dependency graph of sub-tasks, dispatches each to the appropriate worker, reviews the result, and on failure returns one round of critique before accepting it. Results accumulate in a shared store that later tasks read from. Algorithm 1 summarizes the loop; the review step realizes a Reflexion-style self-check [20]. If the Standards Specialist asserts a wrong deprecation date, the supervisor rejects it and forces a re-read of the retrieved NIST text.

Algorithm 1 Quantigence coordination loop

Require: Query $\mathcal{Q}$, worker roles $\mathcal{A}$, tools $\mathcal{T}$

Ensure: Report $\mathcal{R}$

```

1:  $Plan \leftarrow \text{SUPERVISOR.DECOMPOSE}(\mathcal{Q})$ 
2:  $\mathcal{M} \leftarrow \emptyset$  ▷ shared memory
3: for task  $\tau$  in dependency order of  $Plan$  do
4:    $c \leftarrow$  findings in  $\mathcal{M}$  for  $\tau$ 's dependencies
5:    $r \leftarrow \text{AGENTLOOP}(A_\tau, \tau, c, \mathcal{T})$  ▷ worker calls tools until it answers
6:   if  $\text{SUPERVISOR.REVIEW}(r) = \text{FAIL}$  then
7:      $r \leftarrow \text{AGENTLOOP}(A_\tau, \tau, c, \mathcal{T}; \text{critique})$ 
8:   end if
9:    $\mathcal{M} \leftarrow \mathcal{M} \cup \{r\}$ 
10: end for
11:  $\mathcal{R} \leftarrow \text{SUPERVISOR.SYNTHESIZE}(\mathcal{Q}, \mathcal{M})$ 
12: return  $\mathcal{R}$ 

```

3.3 Tool Layer

Each worker calls tools through the model's native function-calling interface; llama.cpp constrains the generated call to the tool's JSON schema, and we validate arguments and retry on error. Three tools ground the agents: (i) arXiv search for literature, (ii) NVD lookup for CVEs, and (iii) a BM25 index over the NIST PQC PDFs (FIPS 203/204/205 and IR 8547), built once and cached. A fourth tool computes QARS (§4). The model's training-data cutoff is thus bridged by live retrieval rather than trusted from memory. The Model Context Protocol [1] is a natural deployment path for these tools but is not required by our implementation.

4 The Quantum-Adjusted Risk Score

We adopt the QARS model of Grigaliūnas and Brūzgienė [6], which scores an asset over timeline, sensitivity, and exposure, and make it automatable. Mosca's inequality states an asset is at risk when

$$X + Y > Z, \tag{1}$$

where X is migration time, Y is the required confidentiality lifetime, and Z is the time until a CRQC. We define the continuous *urgency ratio* $r(a) = (X + Y)/Z$; $r > 1$ is a Mosca violation.

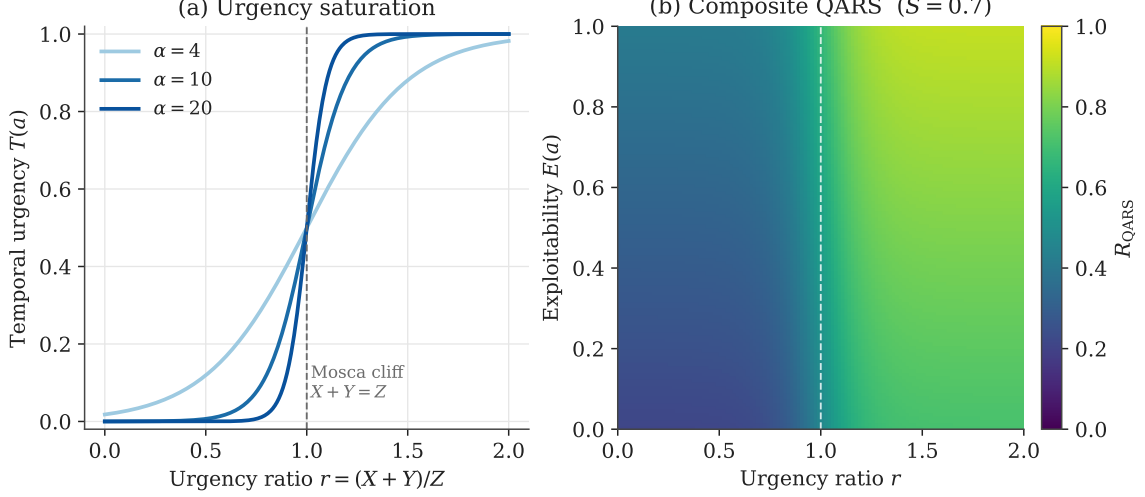


Figure 2: The QARS urgency model. (a) The temporal urgency factor $T(a)$ saturates at the Mosca boundary $r = 1$; larger α sharpens the cliff. (b) The composite score over urgency ratio and exploitability at fixed sensitivity $S = 0.7$: time dominates, but sensitivity and exploitability shift the score away from the boundary.

Because risk near the boundary rises sharply, we map r through a sigmoid to a *temporal urgency factor*

$$T(a) = \frac{1}{1 + e^{-\alpha(r(a)-1)}}, \quad (2)$$

with steepness α (default $\alpha = 10$), so $T = 0.5$ exactly at the boundary and saturates rapidly beyond it (Figure 2a). The composite score combines urgency with normalized sensitivity $S(a)$ and exploitability $E(a)$:

$$R_{QARS}(a) = w_T T(a) + w_S S(a) + w_E E(a), \quad w_T + w_S + w_E = 1, \quad (3)$$

defaulting to the urgency-biased weights $w_T=0.5, w_S=0.3, w_E=0.2$ (Figure 2b). The Standards Specialist estimates Z from community and NIST timelines, the Cryptographic Analyst estimates X from primitive complexity, and the Threat Modeler sets E from CVE severity, so the score is grounded in retrieved evidence. The sigmoid mapping and this agent-driven calibration are our additions; the three-factor decomposition and the QARS name are due to [6].

5 Hardware and Execution Model

A design goal is that useful agentic analysis should not require datacenter hardware. We target an 8 GB VRAM budget, representative of commodity and older GPUs, and serialize agent execution so a single model instance is resident at a time. We use Qwen3.5-9B [15] quantized to 4-bit (Q4_K_M GGUF), served by llama.cpp [5]. Its hybrid attention keeps the key-value cache small, so weights plus an 8k-token context fit comfortably: the measured peak footprint is 5.8 GB (Table 1). Because only one model is resident, “parallel” agents are a reasoning abstraction realized by swapping system prompts and injecting the relevant slice of shared memory, not by concurrent execution. Quantization can degrade quality; we therefore report Q4 results directly and treat the tool-calling reliability of a quantized model as a measured quantity rather than an assumption.

Table 1: Measured memory footprint, Qwen3.5-9B at 4-bit on a single GPU (8k context).

Component	Precision	Notes
Model weights (Qwen3.5-9B)	4-bit (Q4_K_M)	5.7 GB on disk
KV cache + activations	mixed	small; hybrid attention
System / display overhead	n/a	shared GPU
Measured peak VRAM		5.8 GB (within the 8 GB budget)

6 Evaluation

6.1 Benchmark and Metrics

We built a benchmark of 40 queries in four categories of ten: *standards and timelines* (ground truth from the NIST texts), *algorithm analysis* (known key sizes, security categories, and breaks such as SIKE), *vulnerability lookup* (real CVEs), and *risk assessment* (scenarios with a specified X, Y, Z, S, E whose exact QARS is the answer). Each query has a machine-checkable check (keyword, regex, or numeric-proximity for QARS). We report:

- **Accuracy:** fraction passing the deterministic check.
- **Citation validity:** of the arXiv/CVE identifiers a system *states*, the fraction that resolve to real records (the complement is a hallucination rate).
- **Latency and peak VRAM:** wall-clock and memory on the target GPU.

6.2 Ablation

To separate the contribution of tools from that of multi-agent structure, we evaluate three conditions on the same 9B model: (1) *zero-shot* (no tools, no roles), (2) *single agent + tools* (one generalist with the tool loop), and (3) *Quantigence* (full supervisor/worker). Table 2 and Figure 3 report the results.

Table 2: Ablation on the 40-query benchmark (Qwen3.5-9B, 4-bit). Accuracy is against machine-checkable ground truth; citation validity is over stated identifiers.

Condition	Accuracy	Citation validity	Avg. latency	Peak VRAM
Zero-shot	57%	85%	8 s	5.8 GB
Single agent + tools	98%	100%	6 s	5.8 GB
Quantigence	95%	100%	59 s	5.8 GB

What drives the gain. The dominant factor is tool access: grounding the model in the NVD and the NIST index is what lets it answer the vulnerability and risk categories at all, and it eliminates the citation hallucinations present in the zero-shot condition. Crucially, this benchmark is the *wrong* test for multi-agent orchestration: every query has a single checkable answer, so decomposing it into sub-tasks and re-synthesizing can only add cost and risk dropping the exact fact the check looks for. A single tool-using agent is therefore expected to match or beat the full system here, and it does. This is a useful negative result (do not orchestrate an atomic lookup), but it says nothing about the setting orchestration is designed for, which we test next.

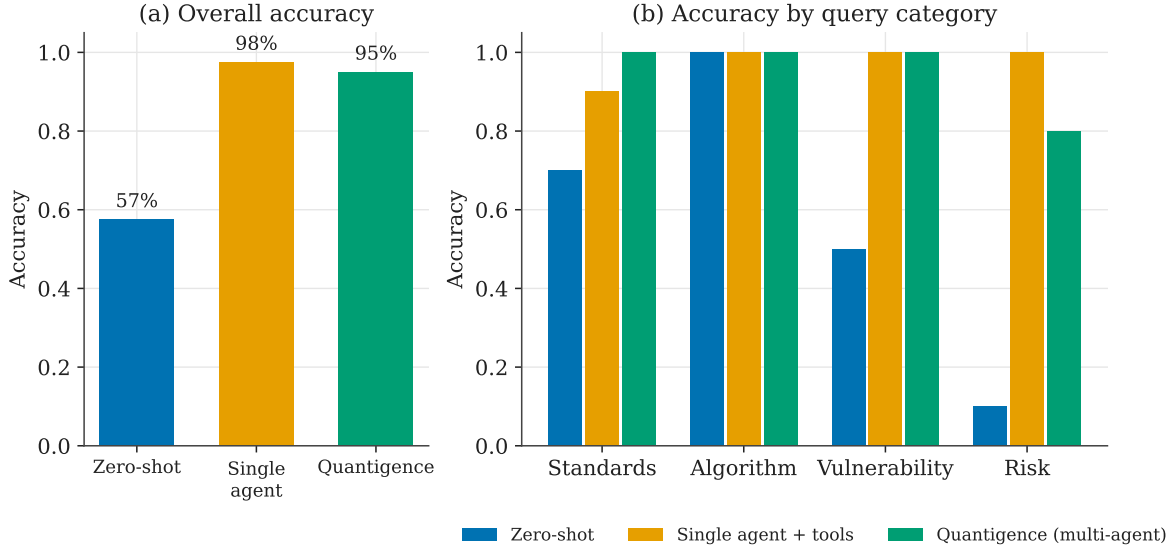


Figure 3: Ablation results. (a) Overall accuracy by condition. (b) Accuracy by query category. The largest gains come from tool access, which lifts the retrieval-bound categories (standards, vulnerability) and the computation-bound risk category that a zero-shot model cannot reliably answer from memory.

6.3 Complex, Multi-Faceted Tasks

Multi-agent research systems are built for, and evaluated on, complex tasks that require breadth and multiple perspectives, using coverage and quality rather than exact match [2]. We therefore built a second benchmark of ten multi-faceted queries, each combining several domains, for example: “assess a system using RSA-2048 and ECDSA with a 20-year confidentiality need: name the standardized replacements for key establishment and signatures, the NIST IR 8547 deadlines, an implementation vulnerability class, and a computed QARS with the Mosca verdict.” Each query carries a rubric of required sub-points (five to six each, 50 in total). We report *coverage* (the fraction of rubric points an answer correctly addresses, scored deterministically) and *quality* (an independent judge model’s rating of comprehensiveness and correctness, normalized to [0, 1]). The judge is a larger, separate model (14B) than the 9B system under test, to avoid self-preference. Table 3 and Figure 4 compare the three conditions.

Here the picture reverses. The multi-agent system covers 89% of the required sub-points, against 78% for the single agent and 76% zero-shot: by assigning each facet to a specialist and synthesizing, it addresses parts of the question a single pass tends to drop. Coverage is deterministic and therefore the load-bearing result; the independent judge also rates the multi-agent answers highest (0.99 vs. 0.94), though its ratings are compressed and less discriminating. The cost is latency, roughly 59 s per query against 6 s, so the trade is coverage for time, which is the right trade for a report-generation task and the wrong one for a lookup.

Table 3: Complex-task evaluation (ten multi-faceted queries, 50 rubric points). Coverage is deterministic; quality is judged by an independent model.

Condition	Coverage	Judged quality	Avg. latency
Zero-shot	76%	0.92	8 s
Single agent + tools	78%	0.94	6 s
Quantigence	89%	0.99	59 s

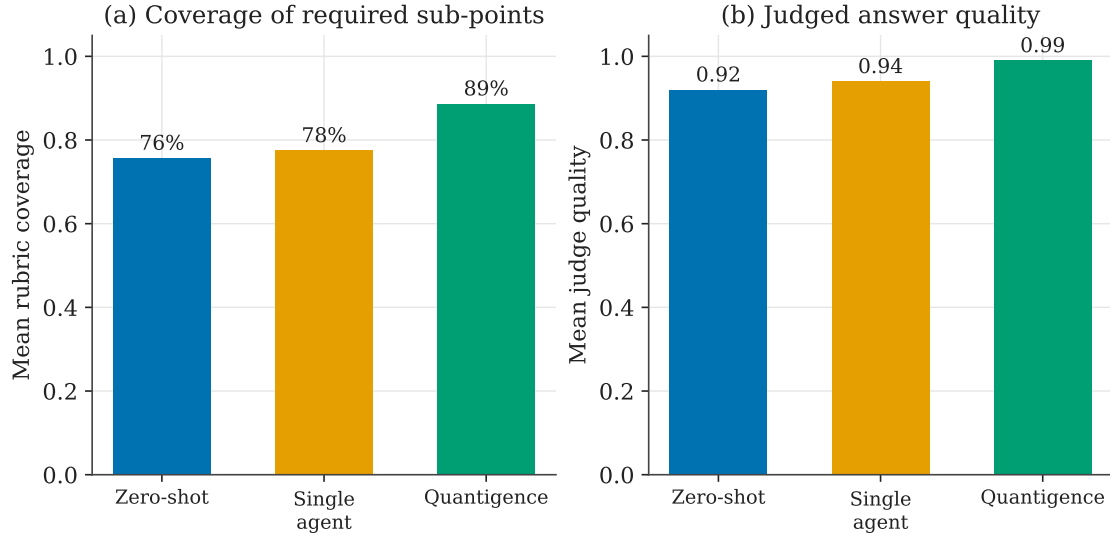


Figure 4: Complex-task results. On multi-faceted queries (the setting orchestration targets), the supervisor/worker decomposition markedly improves rubric coverage (a) over a single agent, and is preferred by an independent judge (b), by assigning each facet to a specialist and synthesizing rather than covering everything in one pass. Contrast with the atomic benchmark (Figure 3), where the same decomposition does not help.

6.4 Robustness to Corpus Poisoning

We inject adversarial documents into the retrieval corpus: false claims (e.g., “ML-KEM has been broken”) keyword-stuffed to be retrieved, carrying embedded prompt-injection payloads (“ignore previous instructions”). Over eight target facts we separate the retrieval and generation conditions, following [3, 29]. Our three defenses are a source hierarchy (NIST-signed sources outrank untrusted text), cross-source consensus, and input sanitization of injection patterns.

Retrieval condition. The poisoning is effective at the retrieval layer: with no defense, a mean of 2.25 of the top-5 chunks are poison and a poison document is ranked first for 62% of targets. The source-hierarchy defense demotes untrusted text and cuts this to 0.75 poison chunks in the top-5, with no poison ever ranked first (Figure 5a).

Generation condition. Despite the poisoned retrieval, the end-to-end system is comparatively robust: an LLM judge (negation-aware) finds the final answer adopts the false claim in only 12% of cases with no defense, and the combined defenses leave this unchanged at 12% while utility stays at 88% (Figure 5b). At this small scale the defenses’ measurable effect is at the retrieval layer, not generation: the resilience comes from grounding many authoritative NIST passages alongside the poison and from the supervisor’s review, not from the defenses alone. This is an argument for the architecture, and a reminder that retrieval poisoning can succeed even when generation does not. A weaker base model, or a higher poison ratio, would rely on the retrieval-layer defense far more.

6.5 Model Scale

We repeat the Quantigence condition over the full 40-query benchmark with a 4B, 9B, and 14B model to map the accuracy-memory trade-off (Figure 6). Within the same Qwen3.5 generation, accuracy rises from the 4B to the 9B model. The 14B point is a different, earlier generation (Qwen3-14B; the Qwen3.5 family has no 14B model), so its accuracy is not a clean size comparison and we include it only for the memory axis: the 4B and 9B models fit the 8 GB budget while the 14B exceeds it, marking the ceiling of the “single commodity GPU” regime.

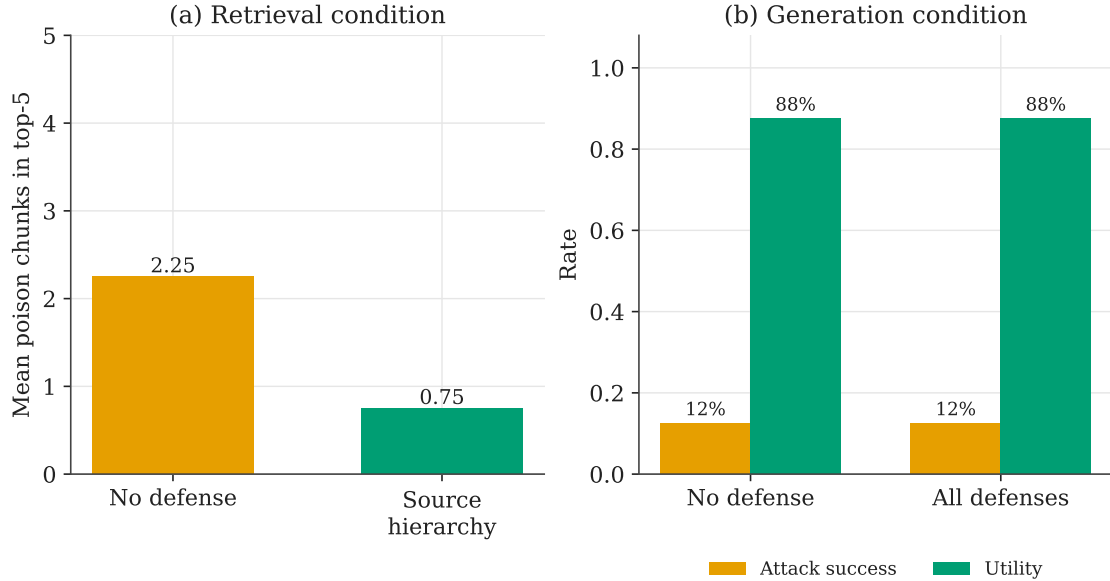


Figure 5: Poisoning robustness. (a) Retrieval condition: mean poison chunks in the top- k , without defenses vs. with the source hierarchy, which demotes untrusted text below NIST-signed evidence. (b) Generation condition: end-to-end attack success and utility, without vs. with all defenses. Retrieval is poisoned, but the answer rarely adopts the false claim.

6.6 Limitations

Our evaluation has clear limits. The benchmark is authored by us and is modest in size; it favors questions with checkable answers and under-represents open-ended synthesis. We do not compare against human experts, so we make no claim about replacing them; Quantigence is decision support with a human in the loop. The atomic checks are keyword- and regex-based, which credit an answer for stating the correct fact and are therefore lenient, so the near-ceiling atomic scores should be read as “states the right fact” rather than as strict grading. Where a judge model is used it may share biases with the system under test. Timing is from a single machine and is not a controlled performance study. Finally, static defenses are known to be weakened by adaptive attacks [25]; our poisoning results should be read as a lower bound on the threat, not a guarantee.

7 Threat Model for the AI Analyst

Delegating security analysis to an agent creates a new attack surface: an adversary’s best move is to deceive the analyst. *Adversarial information poisoning*, flooding preprint servers or blogs with false claims or embedding prompt-injection payloads into retrieved documents, reliably pollutes what the agent reads, as §6.4 shows: poisoned documents dominate the top- k unless demoted. That the final answer usually resists is due to grounding in authoritative sources and the supervisor’s review, not to any guarantee; the defenses (source hierarchy, cross-source consensus, input sanitization) reduce but do not eliminate the risk. This is why Quantigence is designed to triage and support rather than to act autonomously.

8 Conclusion

Quantigence shows that structured, tool-grounded multi-agent analysis of the post-quantum transition is feasible on a single commodity GPU, and that both tool access and agent decomposition measurably improve factual accuracy over a zero-shot baseline on a benchmark with verifiable an-

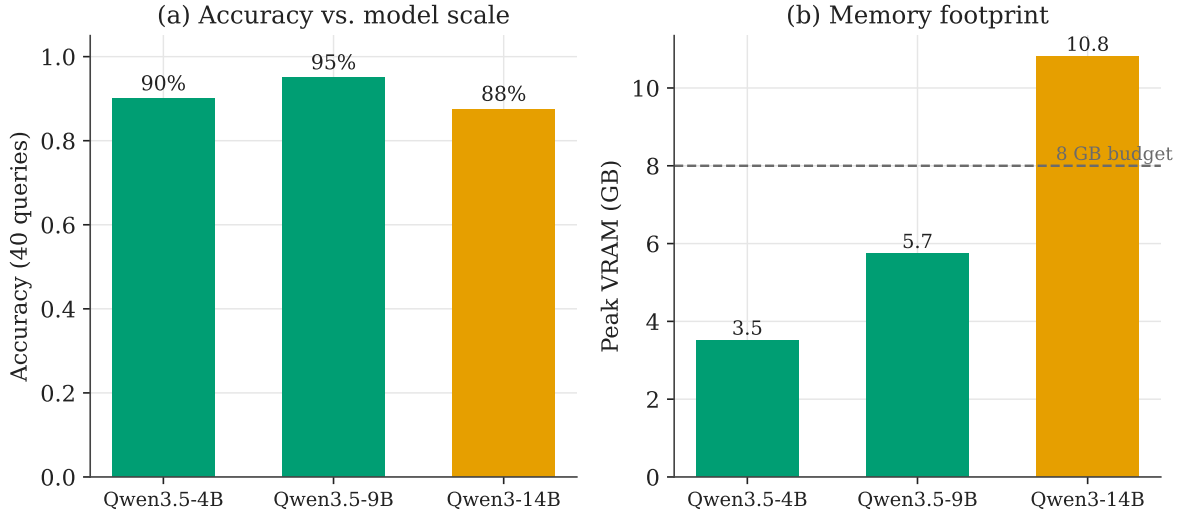


Figure 6: Model-scale ablation (Quantigence, 40 queries). (a) Within the Qwen3.5 generation, accuracy rises from 4B to 9B; the 14B bar is an earlier generation (Qwen3-14B) and is not a clean size comparison. (b) Peak VRAM against the 8GB budget: the 14B model exceeds it, so the 9B model is the practical sweet spot for the target regime.

swers. By automating an existing quantum-risk score and grounding every agent in authoritative sources, it turns a sprawling literature into auditable, prioritized decision support. The system is not a replacement for expert judgment, and its reliance on external evidence makes it a target for poisoning that defenses only partly address. We release the implementation and benchmark to support scrutiny and extension.

Reproducibility. Code, the benchmark, the poisoning corpus, and the scripts that regenerate every figure and number in this paper are available at <https://github.com/AbdulmalikDS/quantigence>.

References

- [1] Anthropic. Introducing the model context protocol. <https://modelcontextprotocol.io>, 2024.
- [2] Anthropic. How we built our multi-agent research system. Anthropic Engineering Blog, 2025. <https://www.anthropic.com/engineering/multi-agent-research-system>.
- [3] Edoardo DeBenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate attacks and defenses for llm agents. In *NeurIPS Datasets and Benchmarks Track*, 2024. arXiv:2406.13352.
- [4] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. PentestGPT: Evaluating and harnessing large language models for automated penetration testing. In *33rd USENIX Security Symposium*, pages 847–864, 2024.
- [5] Georgi Gerganov et al. llama.cpp: Llm inference in c/c++. <https://github.com/ggml-org/llama.cpp>, 2024.
- [6] Šarūnas Grigaliūnas and Rasa Brūzgienė. Towards a unified quantum risk assessment. *Electronics*, 14(17):3338, 2025.

- [7] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proc. 28th Annual ACM Symposium on Theory of Computing (STOC)*, pages 212–219. ACM, 1996.
- [8] Víctor Mayoral-Vilches et al. CAI: An open, bug bounty-ready cybersecurity ai. *arXiv preprint arXiv:2504.06017*, 2025.
- [9] Michele Mosca. Cybersecurity in an era with quantum computers: Will we be ready? *IEEE Security & Privacy*, 16(5):38–41, 2018.
- [10] National Institute of Standards and Technology. FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM). Technical Report NIST FIPS 203, U.S. Department of Commerce, August 2024.
- [11] National Institute of Standards and Technology. FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA). Technical Report NIST FIPS 204, U.S. Department of Commerce, August 2024.
- [12] National Institute of Standards and Technology. FIPS 205: Stateless Hash-Based Digital Signature Standard (SLH-DSA). Technical Report NIST FIPS 205, U.S. Department of Commerce, August 2024.
- [13] National Institute of Standards and Technology. Transition to Post-Quantum Cryptography Standards. Technical Report NIST IR 8547 (Initial Public Draft), U.S. Department of Commerce, November 2024. Initial Public Draft; comment period closed 10 Jan. 2025.
- [14] Javier Pallarés de Bonrostro, Ana I. González-Tablas, and María Isabel González Vasco. Empirical evaluation of large language models for migration of code fragments to post-quantum cryptography. *arXiv preprint arXiv:2606.07341*, 2026.
- [15] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [16] Ramon Ribeiro et al. A crypto-agility readiness score (cars) for post-quantum migration. IACR Cryptology ePrint Archive, Report 2026/1467, 2026.
- [17] Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Zicheng Liu, and Emad Barsoum. Agent laboratory: Using llm agents as research assistants. *arXiv preprint arXiv:2501.04227*, 2025.
- [18] Yijia Shao, Yucheng Jiang, Theodore A. Kanell, Peter Xu, Omar Khattab, and Monica S. Lam. Assisting in writing wikipedia-like articles from scratch with large language models. In *Proc. NAACL*, 2024.
- [19] Animesh Shaw. Quantum-safe code auditing: Llm-assisted static analysis and quantum-aware risk scoring for post-quantum cryptography migration. *arXiv preprint arXiv:2604.00560*, 2026.
- [20] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [21] Peter W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *Proc. 35th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 124–134. IEEE, 1994.
- [22] Peiran Wang, Xiaogeng Liu, and Chaowei Xiao. CVE-Bench: Benchmarking llm-based software engineering agent’s ability to repair real-world cve vulnerabilities. In *Proc. NAACL*, pages 4207–4224, 2025.

- [23] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations (ICLR)*, 2023.
- [24] Andy K. Zhang et al. Cybench: A framework for evaluating cybersecurity capabilities and risks of language models. In *International Conference on Learning Representations (ICLR)*, 2025.
- [25] Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. Agent security bench (asb): Formalizing and benchmarking attacks and defenses in llm-based agents. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.02644.
- [26] Lei Zhang. Toward quantum-safe software engineering: A vision for post-quantum cryptography migration. *arXiv preprint arXiv:2602.05759*, 2026.
- [27] Huichi Zhou, Kin-Hei Lee, Zhonghao Zhan, Yue Chen, Zhenhao Li, et al. TrustRAG: Enhancing robustness and trustworthiness in retrieval-augmented generation. *arXiv preprint arXiv:2501.00879*, 2025.
- [28] Yuxuan Zhu, Antony Kellermann, Dylan Bowman, et al. CVE-Bench: A benchmark for ai agents’ ability to exploit real-world web application vulnerabilities. In *Proc. 42nd International Conference on Machine Learning (ICML)*, 2025. arXiv:2503.17332.
- [29] Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models. In *34th USENIX Security Symposium*, pages 3827–3844, 2025. arXiv:2402.07867.